# AAE-E2001 - Computational Fluid Dynamics D

Assignment 3: Flat Plate

Mohammad-Reza Alibakhshian

February 2026

## Contents

**1 Theory Exercise** **2**

**2 Simulation exercise** **3**

2.1 Physics Identification . . . 3
2.2 Mathematical Equations & Physics Interpretation . . . 3
2.3 Objectives, Feasibility, and Time-Constraints . . . 4
2.4 Numerical Method and Modeling Assumptions . . . 4
2.5 Geometry and Mesh Generation . . . 4
2.6 Computing . . . 5
2.7 Visualization and Post-Processing . . . 5
2.8 Validation and Verification, Reference Data . . . 6
2.9 Questions . . . 7
2.9.1 What dimensions of the domain did you choose and why? What are the cell counts in all spatial directions? . . . 7
2.9.2 How did you determine whether the solution reached steady-state/converged? 8
2.9.3 What are the main sources of discrepancies with Blasius' solution in your case? . . . 8
2.9.4 Why was there a small buffer distance in the mesh before the no-slip plate? 8
2.9.5 What is the maximum velocity magnitude in your domain? If you notice any increase, explain the reason? . . . 8
2.9.6 What is a "passive scalar"? How can you justify the use of a passive scalar for the temperature in this assignment? . . . 9

**3 Appendix 1: Matlab Code for Plotting** **10**

# 1 Theory Exercise

The incompressible momentum equation written in index (Einstein) notation is:

$$\frac{\partial u_i}{\partial t} + u_j \frac{\partial u_i}{\partial x_j} = -\frac{1}{\rho}\frac{\partial p}{\partial x_i} + \frac{1}{\rho}\frac{\partial \tau_{ij}}{\partial x_j}, \tag{1}$$

In Eq. 1, Einstein summation is applied over any index that appears twice in a term; such anindex is called a *dummy index*. For example, in the convective term $u_j\, \partial u_i/\partial x_j$the index $j$ is a dummy index, implying summation over $j = 1, 2, 3$. A dummy index may be renamed (e.g. $j$) without changing the meaning of the expression, provided the replacement does not conflict with otherindices. In contrast, the index $i$ is a *free index* that specifies the velocity component equation(e.g. $i = 1, 2, 3$ corresponding to $u, v, w$).
In Eq. 1 the Newtonian stress tensor for a fluid with constant dynamic viscosity $\mu$ isdefined as:

$$\tau_{ij} = \mu \left( \frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right) \tag{2}$$

**Convective term** Expanding the summation over the index $j$ gives:

$$u_j \frac{\partial u_i}{\partial x_j} = \sum_{j=1}^{3} u_j \frac{\partial u_i}{\partial x_j} = u\frac{\partial u_i}{\partial x} + v\frac{\partial u_i}{\partial y} + w\frac{\partial u_i}{\partial z} \tag{3}$$

where $(u_1, u_2, u_3) = (u, v, w)$ and $(x_1, x_2, x_3) = (x, y, z)$.

**Viscous term** Substituting Eq. (2) into the viscous term of Eq. (1) yields

$$\frac{1}{\rho}\frac{\partial \tau_{ij}}{\partial x_j} = \frac{1}{\rho}\frac{\partial}{\partial x_j}\left[\mu\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right)\right] \tag{4}$$

$$= \frac{\mu}{\rho}\left(\frac{\partial^2 u_i}{\partial x_j \partial x_j} + \frac{\partial}{\partial x_i}\frac{\partial u_j}{\partial x_j}\right). \tag{5}$$

For incompressible flow,

$$\boldsymbol{\nabla} \cdot \boldsymbol{u} = \frac{\partial}{\partial x_i}(u_j)\boldsymbol{e_i}\boldsymbol{e_j} = \frac{\partial}{\partial x_i}(u_j)\delta_{ij} = \frac{\partial u_i}{\partial x_i} = \frac{\partial u_j}{\partial x_j} = 0 \tag{6}$$

and therefore the second term in Eq. 5 vanishes. Introducing the kinematic viscosity $\nu = \mu/\rho$, the viscous term reduces to:

$$\frac{1}{\rho}\frac{\partial \tau_{ij}}{\partial x_j} = \nu\frac{\partial^2 u_i}{\partial x_j \partial x_j} = \nu\sum_{j=1}^{3}\frac{\partial^2 u_i}{\partial x_j \partial x_j} = \nu\left(\frac{\partial^2 u_i}{\partial x^2} + \frac{\partial^2 u_i}{\partial y^2} + \frac{\partial^2 u_i}{\partial z^2}\right) \tag{7}$$

Substituting these values into Eq. 1:

$$\frac{\partial u_i}{\partial t} + u\frac{\partial u_i}{\partial x} + v\frac{\partial u_i}{\partial y} + w\frac{\partial u_i}{\partial z} = -\frac{1}{\rho}\frac{\partial p}{\partial x_i} + \nu\left(\frac{\partial^2 u_i}{\partial x^2} + \frac{\partial^2 u_i}{\partial y^2} + \frac{\partial^2 u_i}{\partial z^2}\right) \tag{8}$$

**Component form of the momentum equations** Setting $i = 1, 2, 3$ in Eq. (8) yields the Caresian component equations.

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} + w\frac{\partial u}{\partial z} = -\frac{1}{\rho}\frac{\partial p}{\partial x} + \nu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}\right), \tag{9}$$

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} + w\frac{\partial v}{\partial z} = -\frac{1}{\rho}\frac{\partial p}{\partial y} + \nu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2}\right), \tag{10}$$

$$\frac{\partial w}{\partial t} + u\frac{\partial w}{\partial x} + v\frac{\partial w}{\partial y} + w\frac{\partial w}{\partial z} = -\frac{1}{\rho}\frac{\partial p}{\partial z} + \nu\left(\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2}\right). \tag{11}$$

These equations correspond to the expanded form of the incompressible Navier–Stokes equations in Cartesian coordinates.

# 2 Simulation exercise

## 2.1 Physics Identification

The physical problem considered in this simulation is the laminar flow of an incompressible Newtonian fluid over a flat plate with a sharp leading edge. A uniform freestream with constant velocity $U$ enters the computational domain and develops a boundary layer along the plate due to the no-slip condition imposed at the wall. The flow is characterized by a low-to-moderate Reynolds number based on the distance from the leading edge,

$$Re_x = \frac{Ux}{\nu} \tag{12}$$

which is restricted to $Re_x < 10^6$ to ensure that the boundary layer remains laminar throughout the region of interest. Under these conditions, the flow is dominated by viscous effects near the wall and inertial effects in the freestream, resulting in the formation and growth of a laminar boundary layer.

In addition to the momentum transport, heat transfer is considered by imposing a constant wall temperature at the flat plate, while the incoming flow enters at a uniform freestream temperature. Thermal transport occurs via convection within the boundary layer and diffusion normal to the wall. The temperature field does not influence the momentum equations and is therefore treated as a passive scalar, which is justified by the incompressible flow assumption and the absence of buoyancy or temperature-dependent material properties.

The flow is assumed to be steady, two-dimensional in its mean behavior, and free of separation or pressure-gradient effects. These assumptions are valid for a flat plate aligned with the flow and allow direct comparison with the classical Blasius solution for velocity and the Pohlhausen solution for the temperature field. The simplicity of the physical configuration makes this case well suited for validation of numerical methods and assessment of mesh resolution requirements in boundary-layer simulations.

## 2.2 Mathematical Equations & Physics Interpretation

The flow is governed by the incompressible Navier–Stokes equations, consisting of the continuity equation,

$$\boldsymbol{\nabla} \cdot \mathbf{V} = 0 \tag{13}$$

and the momentum equations,

$$\frac{\partial \mathbf{V}}{\partial t} + (\mathbf{V} \cdot \boldsymbol{\nabla})\mathbf{V} = -\frac{1}{\rho}\boldsymbol{\nabla} p + \nu \boldsymbol{\nabla}^2 \mathbf{V} \tag{14}$$

where $\mathbf{V}$ is the velocity vector, $p$ is the kinematic pressure, and $\nu$ is the kinematic viscosity.

Since the objective of this study is to obtain the steady-state solution, all time-derivative terms are assumed to vanish, i.e. $\partial(\cdot)/\partial t = 0$. Under this assumption, the governing equations reduce to a balance between convective, pressure, and viscous effects.

Heat transfer is modeled using a scalar transport equation for the temperature field,

$$\frac{\partial T}{\partial t} + \mathbf{V} \cdot \boldsymbol{\nabla} T = \alpha \boldsymbol{\nabla}^2 T \tag{15}$$

The thermal diffusivity is defined as $\alpha = \nu/Pr$, where $\nu$ is the kinematic viscosity and $Pr$ is the Prandtl number. In this study, a constant Prandtl number of $Pr = 0.7$ is used, together with a kinematic viscosity of $\nu = 1.5 \times 10^{-5}\,\mathrm{m}^2/\mathrm{s}$, resulting in a thermal diffusivity of $\alpha \approx 2.14 \times 10^{-5}\,\mathrm{m}^2/\mathrm{s}$. This value is representative of air under standard conditions and is consistent with the assumptions underlying the Pohlhausen similarity solution used for validation.

Physically, the continuity equation enforces mass conservation, while the momentum equation represents the balance between inertial, pressure, and viscous forces within the developing boundary layer. The scalar transport equation describes the balance between convective and diffusive heat transport, leading to the formation of a thermal boundary layer adjacent to the heated wall.

## 2.3 Objectives, Feasibility, and Time-Constraints

The primary objective of this simulation is to compute the steady-state laminar boundary-layer flow over a flat plate and to assess the accuracy of the numerical solution by comparison with analytical reference solutions. Specifically, the velocity field is compared against the Blasius solution, while the temperature field is compared against the Pohlhausen solution for a constant wall temperature.

The problem is computationally feasible due to its simple geometry, laminar flow regime, and steady-state formulation. The absence of turbulence modeling and the use of constant fluid properties significantly reduce the computational cost. Additionally, the availability of analytical solutions provides clear benchmarks for validation without requiring experimental data.

Since the objective is to obtain a steady-state solution, no physical time marching is performed. Instead, the governing equations are solved iteratively using a steady-state algorithm, and the computation is continued until the solution converges. Convergence is assessed by monitoring the residuals of the solved variables, and the simulation is terminated once the residual errors fall below a specified tolerance.

## 2.4 Numerical Method and Modeling Assumptions

The simulations are performed using the **foamRun** application with the **incompressibleFluid** solver. A steady-state formulation is employed, and the time-derivative terms are discretized using a **steadyState** scheme.

Spatial gradients are discretized using the **Gauss linear scheme**, while the convective terms in the momentum and temperature equations are discretized using **bounded linear upwind schemes**. Pressure–velocity coupling is handled using the **SIMPLE** algorithm, which is suitable for steady-state incompressible flow simulations.

The solution is obtained iteratively until convergence is reached. Residual tolerances of $10^{-6}$ are prescribed for pressure, velocity, and temperature. Under-relaxation is applied with a factor of 0.3 for pressure and 0.7 for the velocity equations to ensure numerical stability and smooth convergence towards the steady-state solution.

## 2.5 Geometry and Mesh Generation

The computational domain represents a two-dimensional flat-plate configuration, as shown in Fig. 1. The domain extends from $x = -0.5$ upstream of the leading edge to $x = 1.1$ downstream,

providing sufficient distance for flow development before the plate and for minimizing outlet boundary effects. In the wall-normal direction, the domain spans $0 \leq y \leq 0.5$, which is large enough to fully contain the viscous and thermal boundary layers. A small thickness is used in the spanwise direction ($-0.1 \leq z \leq 0.1$), and the front and back boundaries are treated as empty, enforcing a two-dimensional flow assumption.

The mesh is generated using **blockMesh** and consists of two structured hexahedral blocks in the streamwise direction. The upstream block ($-0.5 \leq x \leq 0$) contains $N = 100$ cells and is used to establish a smooth incoming flow. The downstream block ($0 \leq x \leq 1.1$), which includes the flat plate, is discretized with $N_1 = 220$ cells to provide increased resolution along the plate where boundary-layer growth occurs. In the wall-normal direction, $N_2 = 100$ cells are used.

Mesh grading is applied in the wall-normal direction using a grading factor of $G = 200$, resulting in strong refinement near the flat plate to accurately resolve the viscous and thermal boundary layers. Away from the wall, the cell size increases gradually to reduce the total cell count. In the streamwise direction, a mild grading is applied in the downstream block to provide higher resolution near the leading edge, where wall shear stress gradients are largest.

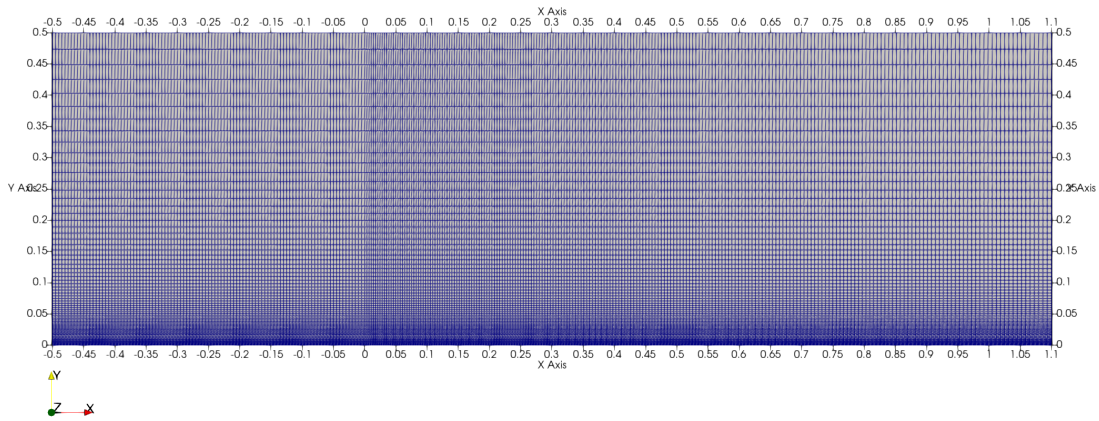


Figure 1: Computational domain and structured mesh for the flat-plate simulation. The mesh is refined in the wall-normal direction near the flat plate to resolve the viscous and thermal boundary layers, while coarser cells are used away from the wall to reduce computational cost.

## 2.6 Computing

The simulations are executed in serial using the **foamRun** utility. A steady-state iterative solution procedure is employed, and the computation is continued until convergence is achieved based on the prescribed residual criteria. Residuals of pressure, velocity, and temperature are monitored throughout the simulation to ensure numerical stability and convergence. The simulation converged after 596 iterations. Owing to the two-dimensional configuration, structured mesh, and laminar flow assumption, the overall computational cost remains low.

## 2.7 Visualization and Post-Processing

Visualization and post-processing are performed using ParaView/paraFoam. The overall development of the boundary layers is inspected using contour plots of velocity magnitude and temperature. In addition, close-up views at the leftmost sampling location are generated to verify that the mesh resolution is adequate near the wall. These close-up figures also include a

ruler indicating the exact sampling line location and its length for profile extraction (see Figs. 2 and 2).

Quantitative post-processing is carried out by extracting wall-normal profiles of velocity and temperature at selected streamwise positions and comparing them with the corresponding Blasius and Pohlhausen similarity solutions. The wall shear stress is also sampled along the plate to compute the local skin-friction coefficient $C_f(x)$ for comparison with the analytical Blasius prediction.

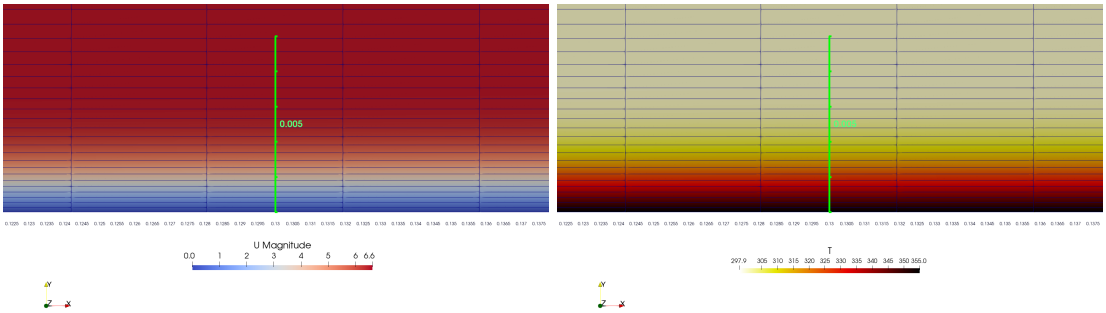


Figure 2: Close-up views of the viscous (left) and thermal (right) boundary layers near the flat plate at the $x \approx 0.13$ m sampling location. The mesh refinement in the near-wall region is visible, and the ruler indicates the location, number of cells and length of the wall-normal sampling line used for extracting the velocity and temperature profiles.

## 2.8 Validation and Verification, Reference Data

Validation of the numerical results is performed by comparison with classical analytical solutions for laminar boundary-layer flow over a flat plate. Specifically, the numerical velocity, skin-friction coefficient and temperature profiles are compared against the Blasius and Pohlhausen similarity solutions.

Figure 3 presents the streamwise velocity profiles as a function of the wall-normal coordinate $y$ at three streamwise locations ($x = 0.1$, 0.4, and 0.8 m). The numerical results show excellent agreement with the corresponding Blasius velocity profiles. The progressive thickening of the boundary layer with increasing streamwise distance is captured accurately, indicating that the viscous effects and near-wall resolution are well resolved.

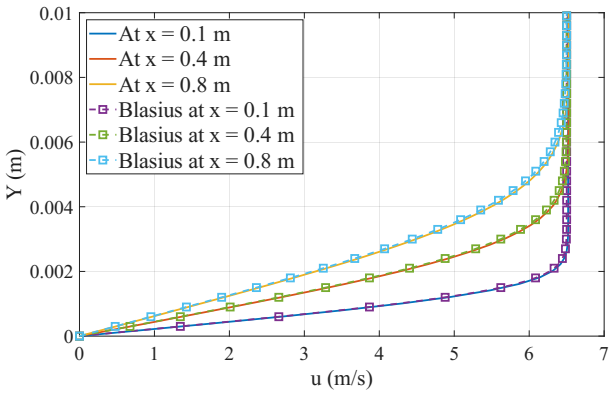


Figure 3: Comparison of numerical and Blasius streamwise velocity profiles at $x = 0.1$, 0.4, and 0.8 m, illustrating the downstream growth of the laminar boundary layer.

Figure 4 shows the temperature profiles at the same streamwise locations, compared with the Pohlhausen similarity solution for a constant wall temperature. The numerical tempera-

ture profiles closely follow the analytical predictions, demonstrating correct resolution of the thermal boundary layer and consistency with the assumed Prandtl number. As expected, the thermal boundary layer thickness increases downstream, following the same trend as the velocity boundary layer.

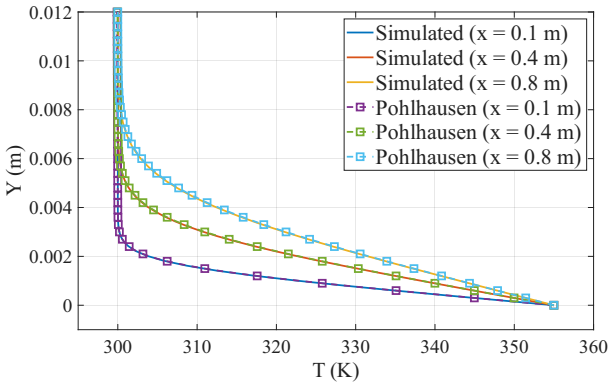


Figure 4: Comparison of numerical and Pohlhausen temperature profiles at $x = 0.1$, 0.4, and 0.8 m, illustrating the downstream development of the thermal boundary layer.

Further verification is provided by the comparison of the local skin-friction coefficient $C_f(x)$ along the plate, shown in Fig. 5. The numerically computed skin-friction coefficient exhibits the expected monotonic decrease with streamwise distance and shows good agreement with the Blasius analytical solution. Minor discrepancies near the leading edge can be attributed to finite mesh resolution and the numerical treatment of the sharp leading-edge region.

Overall, the close agreement between the numerical results and the analytical reference solutions confirms the correctness of the numerical implementation, mesh resolution, and modeling assumptions employed in this study.

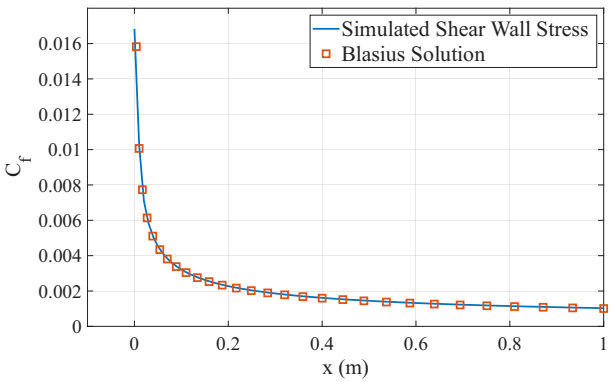


Figure 5: Numerical and Blasius skin-friction coefficient distributions along the flat plate, illustrating the downstream decay of wall shear stress.

## 2.9 Questions

### 2.9.1 What dimensions of the domain did you choose and why? What are the cell counts in all spatial directions?

The domain was selected to avoid boundary-condition interference and to allow smooth inflow development before the plate. It extends from $x = -0.5$ to $x = 1.1$, with the plate starting at $x = 0$. The wall-normal height is $0 \leq y \leq 0.5$, which is sufficient to contain the viscous and thermal boundary layers, and the spanwise thickness is $-0.1 \leq z \leq 0.1$ with **empty** front/back patches (2D setup).

The mesh consists of two structured blocks in $x$: 100 cells for $-0.5 \leq x \leq 0$ and 220 cells for $0 \leq x \leq 1.1$. In the wall-normal direction, $N_2 = 100$ cells are used with strong grading towards the plate, and 1 cell is used in the spanwise direction. Overall, the chosen domain and mesh were sufficient to reproduce the Blasius and Pohlhausen reference solutions.

#### 2.9.2 How did you determine whether the solution reached steady-state/converged?

The solution was considered converged once a steady-state was reached and the residuals of all solved variables decreased below the prescribed tolerance. Convergence was monitored by tracking the residuals of pressure, velocity, and temperature, which were required to fall below $10^{-6}$. In addition, global quantities such as velocity and temperature profiles, as well as the skin-friction coefficient along the plate, were observed to remain unchanged with further iterations. The simulation converged after 596 iterations, indicating that a steady-state solution had been achieved.

#### 2.9.3 What are the main sources of discrepancies with Blasius' solution in your case?

The main sources of discrepancies with the Blasius solution arise from numerical and modeling limitations rather than physical effects. The Blasius solution is derived for an idealized semi-infinite flat plate with a zero-thickness leading edge, whereas the numerical simulation employs a finite computational domain and a discretized leading-edge region. This results in small deviations near the plate leading edge, where velocity gradients are highest.

Additional discrepancies can be attributed to finite mesh resolution, particularly very close to the wall, and to the numerical discretization of the governing equations. Minor differences may also arise from the treatment of inlet and outlet boundary conditions and from truncation of the domain in the wall-normal direction. Despite these factors, the overall agreement with the Blasius solution is good, indicating that the dominant boundary-layer physics are accurately captured.

#### 2.9.4 Why was there a small buffer distance in the mesh before the no-slip plate?

A small buffer distance was included upstream of the no-slip plate to allow the incoming flow to adjust smoothly from the inlet boundary condition before encountering the sharp leading edge of the plate. This buffer region reduces numerical artifacts and spurious gradients that may arise if the no-slip condition is imposed immediately at the inlet. It also helps stabilize the solution near the leading edge and ensures that the development of the boundary layer along the plate is governed by the physics of the flow rather than by the inlet boundary condition.

#### 2.9.5 What is the maximum velocity magnitude in your domain? If you notice any increase, explain the reason?

The maximum velocity magnitude in the computational domain was obtained using the OpenFOAM post-processing command **foamPostProcess -func "cellMaxMag(mag(U))"**. The maximum value was $U_{max} = 6.94$ m/s (more precisely, $\text{mag}(U) = 6.93514$) located at $x = -0.0025$, $y = 6.54495 \times 10^{-5}$, and $z = 0$, i.e. close to the leading-edge region of the plate. In ParaView, the corresponding value appears lower (e.g. $\approx 6.62$ m/s) because ParaView typically displays an interpolated point value, whereas the OpenFOAM utility reports a cell-centred maximum.

The slight increase above the imposed freestream velocity ($U = 6.5$ m/s) is attributed to numerical effects in the vicinity of the sharp leading edge, where velocity gradients are strongest and discretization may introduce a small local overshoot. This increase is localized and does not

significantly affect the overall boundary-layer development or the agreement with the Blasius reference solution.

In addition, although the inlet is prescribed as 6.5 m/s, the maximum freestream velocity increases slightly downstream (approximately 6.52 m/s at $x = 0.1$ to 6.63 m/s at $x = 0.8$), as shown in Fig. 6. This trend is consistent with the growth of the boundary layer, which reduces the effective flow area; with symmetry boundaries preventing mass flux through the top and bottom boundaries, the outer flow accelerates slightly to satisfy overall mass conservation.

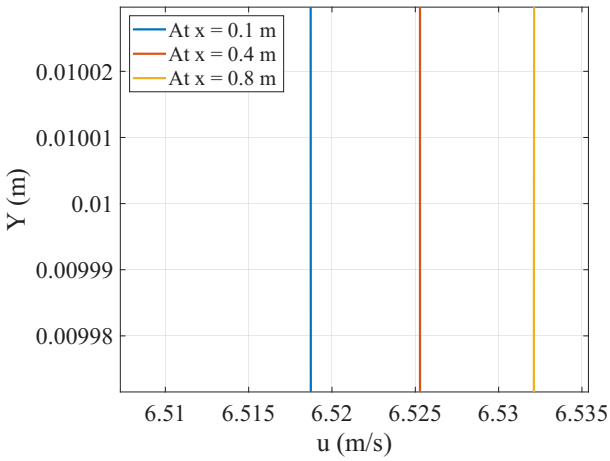


Figure 6: Freestream computed velocity at different sections.

#### 2.9.6 What is a "passive scalar"? How can you justify the use of a passive scalar for the temperature in this assignment?

A passive scalar is a scalar quantity that is transported by the flow through convection and diffusion but does not influence the velocity or pressure fields. In other words, while the scalar is affected by the flow, it does not feed back into the momentum equations.

In this assignment, the temperature is treated as a passive scalar because the flow is assumed to be incompressible with constant fluid properties and no buoyancy effects. The temperature field therefore does not modify the density, viscosity, or pressure, and it has no impact on the momentum balance. Under these assumptions, temperature transport is governed solely by advection and thermal diffusion within the boundary layer, making the passive-scalar approximation appropriate and consistent with the classical Pohlhausen solution used for validation.

# 3 Appendix 1: Matlab Code for Plotting

Listing 1: MATLAB code for skin-friction coefficient

```
clc; clear;

%% The default setup
c = [
    0         0.4470    0.7410;  % Blue
    0.8500    0.3250    0.0980;  % Orange
    0.9290    0.6940    0.1250;  % Yellow
    0.4940    0.1840    0.5560;  % Purple
    0.4660    0.6740    0.1880;  % Green
    0.3010    0.7450    0.9330;  % Light Blue
    0.6350    0.0780    0.1840;  % Red
    0.75      0.75      0;       % Olive
    0.75      0         0.75;    % Magenta
    0.25      0.25      0.25;    % Dark Gray
    1         0.6       0.6;     % Light Pink
    0         0.5       0;       % Dark Green
    0.5       0.5       0.5;     % Gray
    1         0.41      0.16;    % Dark Orange
    0         0         0        % Black
    ];
set(groot, ...
    'DefaultAxesFontName',       'Times New Roman', ...
    'DefaultTextFontName',       'Times New Roman', ...
    'DefaultLegendFontName',     'Times New Roman', ...
    'DefaultAxesColorOrder',      c,              ...
    'DefaultAxesXGrid',          'on',            ...
    'DefaultAxesYGrid',          'on',            ...
    'DefaultAxesZGrid',          'on',            ...
    'DefaultAxesBox',            'on',            ...
    'DefaultLegendFontSize',      22,             ...
    'DefaultAxesFontSize',        22,             ...
    'DefaultLineLineWidth',       2, ...
    'DefaultAxesGridAlpha',0.1, ...
    'DefaultFigurePosition',[100 100 1000 730]);

mrkrs = {'o','s','^','d','v','>','<','p','h','x','+'};
lineStyles = {'-','--','-.',':','-'};

%% Read Table and the corresponding calculations
blasiusData = readtable("blasius.dat");
blasiusData.Properties.VariableNames = {'eta','f_eta','f_prime_eta','theta'};

near = readtable('near.dat');           % it is located at the x=0.1
near.Properties.VariableNames = {'y','u','v','w','T'};
x_near = 0.1;

mid = readtable('mid.dat');             % it is located at x = 0.4
mid.Properties.VariableNames = {'y','u','v','w','T'};
x_mid = 0.4;

far = readtable('far.dat');             % it is located at x = 0.8
far.Properties.VariableNames = {'y','u','v','w','T'};
x_far = 0.8;

```

```
wallShear = readtable('wallShear.dat');
wallShear.Properties.VariableNames = {'x','shear_x','shear_y','shear_z'};

% Calculation of variables
nu = 1.5e-5;    % m^2/s
U = 6.5;        % m/s
T_e = 300;      % K, freestream temperature
T_w = 355;      % K, wall temperature
y = linspace(0,0.02,201);
Pr = 0.7;       % Prandtl Number
D = nu/Pr;      % This is the diffusivity number (m^2/s)

eta =@(x,y) y .* sqrt(U ./ (2 .* nu .* x));
u_blasius = @(u,x,y) u .* interp1(blasiusData.eta, blasiusData.f_prime_eta,eta(x,y),'linear','extrap');
theta_pohlhausen = @(x,y)  interp1(blasiusData.eta, blasiusData.theta,eta(x,y),'linear','extrap');
T_pohlhausen =@(x,y) T_e + (T_w -T_e) .* theta_pohlhausen(x,y);

x = linspace(0.001,1,1000);
dudy_blas = u_blasius(U,x,y(2)) ./ y(2);
C_f_blasius = 2 .* nu .* dudy_blas ./ (U^2);

%% Figure 1 blasius plot
figure(1); clf; hold on;
% set(gcf, 'Position', [100 100 1000 730]);
plot(blasiusData.eta, blasiusData.f_eta,'DisplayName','f(\eta)');
plot(blasiusData.eta, blasiusData.f_prime_eta,'DisplayName','f''(\eta)');
plot(blasiusData.eta, blasiusData.theta,'DisplayName','\theta(\eta)');

xlabel('\eta'); title('Blasius Solution of Laminar Boundary Layer');
legend('Location','best'); hold off;
exportgraphics(gcf,'Fig1.jpg');
exportgraphics(gcf,'Fig1.pdf','ContentType','vector');

%% Figure 2
figure(2); clf; hold on;

plot(near.u, near.y,'DisplayName','At x = 0.1 m');
plot(mid.u, mid.y,'DisplayName','At x = 0.4 m');
plot(far.u, far.y,'DisplayName','At x = 0.8 m');

plot(u_blasius(U,x_near,y),y,'s--','DisplayName','Blasius at x = 0.1 m','MarkerIndices',1:3:100,'MarkerSize',10);
plot(u_blasius(U,x_mid,y),y,'s--','DisplayName','Blasius at x = 0.4 m','MarkerIndices',1:3:100,'MarkerSize',10);
plot(u_blasius(U,x_far,y),y,'s--','DisplayName','Blasius at x = 0.8 m','MarkerIndices',1:3:100,'MarkerSize',10);

axis([0 7 0 0.01]); xlabel('u (m/s)'); ylabel('Y (m)');hold off;
legend('Location','northwest','FontSize',25);
exportgraphics(gcf,'Fig2.jpg');
exportgraphics(gcf,'Fig2.pdf','ContentType','vector');

%% Figure 3 Temperature Profile of Laminar Boundary Layer
figure(3); clf; hold on;

plot(near.T, near.y,'DisplayName','Simulated (x = 0.1 m)');
```

```
plot(mid.T, mid.y,'DisplayName','Simulated (x = 0.4 m)');
plot(far.T, far.y,'DisplayName','Simulated (x = 0.8 m)');

% set(gca,'ColorOrderIndex',1);
plot(T_pohlhausen(0.1,y),y,'s--','DisplayName','Pohlhausen (x = 0.1 m)','MarkerIndices',1:3:200,'MarkerSize',10);
plot(T_pohlhausen(0.4,y),y,'s--','DisplayName','Pohlhausen (x = 0.4 m)','MarkerIndices',1:3:200,'MarkerSize',10);
plot(T_pohlhausen(0.8,y),y,'s--','DisplayName','Pohlhausen (x = 0.8 m)','MarkerIndices',1:3:200,'MarkerSize',10);

axis([295 360 -0.001 0.012]);
ax = gca;
ax.YAxis.Exponent = 0;
xlabel('T (K)'); ylabel('Y (m)');hold off;
legend('Location','northeast','FontSize',25);
exportgraphics(gcf,'Fig3.jpg');
exportgraphics(gcf,'Fig3.pdf','ContentType','vector');

%% Figure 4
figure(4); clf; hold on;

% Number of markers you want
nMarkers = 30;
% Create clustered indices near the beginning
idx = unique(round(linspace(1, sqrt(numel(x)), nMarkers).^2));
idx(idx > numel(x)) = [];

plot(wallShear.x, -2 .* wallShear.shear_x ./ U^2,'DisplayName','Simulated Shear Wall Stress','LineWidth',2);
plot(x,C_f_blasius,'s','DisplayName','Blasius Solution','MarkerIndices',idx,'MarkerSize',10);
axis([-0.1 1 0 0.018 ]);


xlabel('x (m)'); ylabel('C_f');
legend('Location','northeast','FontSize',25); hold off;
exportgraphics(gcf,'Fig4.jpg');
exportgraphics(gcf,'Fig4.pdf','ContentType','vector');
```